

TAOD Accelerator — AXI Interface Details

DVCon India 2026 Design Contest — Stage 2B

1. Overview

The TAOD accelerator exposes two AXI interfaces:

- An AXI4-Lite-style slave through which the host (VEGA) reads and writes control/status registers, loads the task-weight LUT, and pushes object records.
- An AXI4 master through which the accelerator autonomously DMA-reads object records from memory and writes the result back.

All probability like values (detector confidences, fused confidence, task weights, temporal stability) use UQ2.14 fixed point: 16 bit unsigned, 2 integer + 14 fractional bits, with the value 1.0 represented by 16384(0x4000). Multiplier products are kept as 32 bit UQ4.28, the per object score is a full 32 bit UQ4.28 value.

1.1 AXI parameters

Parameter	Value	Description
Data width	64 bits	AXI_DATA_W; one 64-bit word per beat.
Write strobe	8 bits	AXI_STRB_W = AXI_DATA_W/8.
ID width	12 bits	AXI_ID_W (slave and master).
Address (decoded)	32 bits	Low 32 bits decoded; VEGA presents 64-bit address.
Base address	0x2006_0000	Low 16 bits select the register / window.

2. Address Map

The low 16 bits of the AXI address select one of three regions:

Region	Offset range	Contents
CSR	0x0000–0x0FFF	Control/status registers.
Weight LUT	0x1000–0x37FF	Task×class relevance weights(1280 entries × 8 bytes).
Object window	0x4000–0x43FF	Object records(64 records × 16 bytes).

3. Control/Status Register Map

Offsets are relative to the base address. Access: W = write, RO = read only, R/W = read write.

Offset	Name	Acc.	Description
0x000	CTRL	W	Control strobes: start, clear, dma_start (see 3.1).
0x008	TASK	R/W	Task ID [3:0] (0–15).
0x010	OBJCNT	R/W	Object count [6:0] (0–64).
0x018	FUSE	R/W	Fusion weights: w1 [15:0], w2 [31:16] (UQ2.14).
0x020	INVFA	R/W	Inverse frame area [31:0] (area-term scale).

Offset	Name	Acc.	Description
0x028	ALPHA	R/W	EMA shift coefficient [3:0].
0x030	DMAOBJ	R/W	DMA object source base address [31:0].
0x038	DMARES	R/W	DMA result destination base address [31:0].
0x040	IRQCTL	W	Interrupt control: irq_en [0], irq_clr [1].
0x0F8	VERSION	RO	Version identifier = 0x7A0D_0001.
0x100	STATUS	RO	Busy / done / IRQ / DMA status (see 3.2).
0x108	RESULT	RO	Winning score / class / index (see 3.3).
0x110	RUNNER	RO	Runner-up score [31:0] (UQ4.28).
0x118	BBOX	RO	Winning bounding box (see 3.4).
0x120	PERF_COMPUTE	RO	Engine compute cycles of last frame (= 6N+2).
0x128	PERF_FRAME	RO	Full-frame cycles incl. DMA read.

3.1 CTRL (0x000, write)

Bit	Name	Description
0	start	Launch a scoring pass in CPU-push mode.
1	clear	Clear engine state / result registers.
2	dma_start	Launch a scoring pass in DMA mode.

3.2 STATUS (0x100, read-only)

Bit	Name	Description
0	eng_busy	Scoring engine is running.
1	done_sticky	A scoring pass has completed (sticky).
2	eng_result_valid	Result registers hold a valid result.
3	irq_pending	Interrupt pending.
4	dma_busy	DMA transfer in progress.
5	dma_done_sticky	DMA frame completed(sticky).

3.3 RESULT (0x108, read only)

Bits	Field	Description
[63:32]	res_score	Winning object score (UQ4.28).
[15:8]	res_class	Winning object class (COCO, 8-bit).
[5:0]	res_idx	Winning object index (0–63).
others	—	Reserved / zero.

3.4 BBOX (0x118, read-only)

Bits	Field	Description
[15:0]	x1	Winning box: left x (pixels).
[31:16]	y1	Winning box: top y.
[47:32]	x2	Winning box: right x.

Bits	Field	Description
[63:48]	y2	Winning box: bottom y.

4. Weight LUT Window (0x1000–0x37FF)

The LUT holds one UQ2.14 relevance weight per pair, 1280 entries total, each occupies 8-byte slot. The entry index and byte offset are:

$$index = task * 80 + class$$

$$byte_offset = 0x1000 + index * 8$$

Write 16-bit weight in low bits of 64-bit data word. Load the table once before scoring, it persists across frames and can be reprogrammed any time.

5. Object Record Format (Object Window 0x4000 – 0x43FF)

Each object is 128-bit record. The window holds up to 64 records. Record i occupies bytes [$i \times 16$.. $i \times 16 + 15$]: The low 64-bit word at offset $i \times 16$, the high 64-bit word at offset $i \times 16 + 8$. The layout of the 128-bit record is:

Bits	Field	Width	Description
[15:0]	x1	16	Bounding box left x (pixels).
[31:16]	y1	16	Bounding box top y.
[47:32]	x2	16	Bounding box right x.
[63:48]	y2	16	Bounding box bottom y.
[79:64]	c1	16	Detector model-1 confidence (UQ2.14).
[95:80]	c2	16	Detector model-2 confidence (UQ2.14).
[103:96]	class	8	COCO class id (0–79).
[111:104]	track	8	Track id (0–255) for temporal EMA.
[112]	valid	1	Object valid flag.
[127:113]	reserved	15	Reserved (write = 0).

The two 64-bit words are:

$$low\ word\ [63:0] = \{ y2, x2, y1, x1 \}$$

$$high\ word\ [127:64] = \{ reserved[14:0], valid, track, class, c2, c1 \}$$

6. AXI4-Lite Slave Interface

The slave is a 64 bit AXI4 Lite interface used for all host initiated access: control/status register read and write, weight LUT loading, and object record push. It uses the standard write channels (AW, W, B) and read channels (AR, R).

- Writes: The host drives AW (address) and W (64-bit data + byte strobes) and the slave responds on B with an OKAY response. The decoded low-16 address selects the register or window slot.
- Reads: The host drives AR and the slave returns the addressed register on R with an OKAY response. Reads of unmapped CSR offsets return 0.
- Object push: Writing the low and high 64-bit words of a record into the object window (Section 5) stores it in the on-chip object buffer.

7. AXI4 Master Interface (DMA)

In DMA mode the accelerator acts as an AXI4 master to move data without per-object host involvement:

- Object read: starting from DMAOBJ (0x030), the master burst-reads object records (two 64-bit beats per 16-byte record) on the AR/R channels into the object buffer, then runs the scoring pass.
- Result write-back: the master writes the result to DMARES (0x038) on the AW/W channels and observes the B response.
- Write ordering: WVALID and WLAST are presented combinational relative to the write handshakes so that WLAST coincides with the final data beat, meeting AXI write-handshake requirements (this corrects an earlier registered-WVALID issue that could stall a compliant slave).

A DMA pass is launched by writing CTRL.dma_start (0x000 bit 2) after programming DMAOBJ, DMARES, OBJCNT and the scoring parameters. Completion is signaled by STATUS.dma_done_sticky and, if enabled then an interrupt.

8. Programming Sequence

8.1 CPU-push mode

1. Load the weight LUT (once): Write each (task, class) weight to the LUT window (Section 4).
2. Program parameters: TASK(0x008), FUSE w1/w2(0x018), INVFA(0x020), ALPHA (0x028), OBJCNT(0x010).
3. Optionally enable interrupts: IRQCTL.irq_en (0x040 bit 0).
4. Push object records: Write low and high words of each record into object window (Section 5).
5. Launch: write CTRL.start (0x000 bit 0).
6. Wait: poll STATUS.done_sticky (0x100 bit 1) or wait for interrupt.
7. Read results: RESULT(0x108), RUNNER(0x110), BBOX(0x118), In optional PERF_COMPUTE(0x120) and PERF_FRAME(0x128).

8. Clear for next frame: Write CTRL.clear(0x000 bit 1), clears interrupt via IRQCTL.irq_clr(0x040 bit 1), if it is used.

8.2 DMA mode

9. Load the weight LUT and program scoring parameters as in steps 1–3 above.
 10. Program DMA addresses: DMAOBJ source(0x030) and DMARES destination(0x038), and OBJCNT(0x010).
 11. Launch: write CTRL.dma_start(0x000 bit 2). The accelerator reads the objects, scores them, and write the result back.
 12. Wait: poll STATUS.dma_done_sticky(0x100 bit 5) or wait for interrupt, then read the result registers or the DMARES location.
-